

מטלה בלמידת מכונה – עיבוד תמונה

סיווג מרובה-מחלקות של תמונות ידיים במשחק אבן, נייר ומספריים (Rock-Paper-Scissors)
אלגוריתם: K-Nearest Neighbors (KNN) | סוג למידה: Classification | מדד איכות: Macro-Average F1

פרטי הסטודנטים

שם	ארבע ספרות אחרונות של ת.ז.	תפקיד בהצגה
ניר י.	1498	הקדמה וטעינת הנתונים (חלק 1); תוצאות, ניתוח ומסקנות (חלק 5)
דניאל צ.	6313	הנדסת מאפיינים (חלק 2)
יהונתן י.	9248	אלגוריתם הלמידה, Grid-Search ואימון (חלקים 3-4)

קישורי ההגשה

תוצר	קישור
Kaggle Dataset ב-Kaggle	https://www.kaggle.com/datasets/sanikamal/rock-paper-scissors-dataset
Repository (GitHub)	https://github.com/NirYair/MLProject
מחברת הקוד (Colab)	https://colab.research.google.com/drive/1zjJZ_78SL8NWFuMHZ40woDPJqcSgt_kKx?usp=sharing
סרטון ההסבר (YouTube)	https://youtu.be/5SF8hW6PRyA

מיפוי בין חלקי המטלה לתאי המחברת

המחברת `RockPaperScissors_Final.ipynb` בנויה מתאים הממוספרים 0 PART עד 5 PART. להלן ההתאמה בין הסעיפים במסמך ההנחיות לבין תאי המחברת, כדי להקל על הבדיקה.

סעיף במטלה	תא במחברת	תוכן
חלק 1 – הקדמה וטעינת נתונים	PART 0, PART 1, PART 2	ייבוא ספריות, הגדרת נתיבים, טעינת Train ו-Test בנפרד
חלק 2 – Feature Engineering	PART 3	Otsu, Bounding-Box Crop, Aspect-Ratio Padding, Scaling
חלק 3 – מימוש אלגוריתם הלמידה	PART 4	KNN, היפר-פרמטרים, <code>fit</code> ו- <code>predict</code>
חלק 4 – אימון לפי פרמטרים שונים	PART 4	Grid-Search, 5-Fold CV, אימון מחדש על כל ה-Trainset
חלק 5 – חיזוי ושערוך איכות	PART 5	חיזוי על ה-Test, ויזואליזציה ו-Macro F1 סופי

0.7738

Macro F1 על ה-Test Set

0.9996

Cross-ב Macro F1
Validation

372

תמונות Test

2,520

תמונות Train

חלק 1 – הקדמה, בעיית הלמידה וטעינת הנתונים

1.1 הסבר על בעיית הלמידה

הבעיה שבחרנו היא בעיית סיווג מרובה-מחלקות (**Multi-Class Classification**) בתחום הראייה הממוחשבת: בהינתן תמונה גולמית של כף יד, על המערכת לקבוע לאיזו משלוש התנועות של המשחק "אבן, נייר ומספריים" היא שייכת – `rock`, `paper` או `scissors`. זוהי בעיית למידה מונחית (**Supervised Learning**): לכל תמונה בסט האימון קיימת תווית אמת הנגזרת משם התיקיה שבה היא נמצאת, והמודל לומד את המיפוי מתמונה לתווית. האתגר המרכזי הוא שהקלט הוא תמונה גולמית של 300×300 פיקסלים בשלושה ערוצי צבע – ייצוג בעל 270,000 ממדים, שרובם המכריע (הרקע, גוון העור, הצללית) אינו רלוונטי להבחנה בין שלוש התנועות. לכן עיקר העבודה מתמקד בהפקת ייצוג קומפקטי שמשמר את צורת היד בלבד.

1.2 הסבר על ה-Dataset

בחרנו ב-Dataset בשם **Rock-Paper-Scissors** מאתר Kaggle (מאת sanikamal). ה-Dataset מורכב מתמונות מרונדרות (CGI) של כפות ידיים בגוונים ובזוויות שונות, על רקע לבן אחיד, בפורמט PNG בגודל 300×300 פיקסלים. הנתונים מגיעים מחולקים מראש לתיקיות `train` ו-`test`, וכל אחת מהן מכילה שלוש תת-תיקיות לפי הקטגוריות. בהתאם להנחיות המטלה, טענו את שני הסטים בנפרד ולא חילקנו אותם מחדש, כדי למנוע זליגת נתונים (Data Leakage).

סט	rock	paper	scissors	סה"כ
Train	840	840	840	2,520
Test	124	124	124	372

שני הסטים **מאוזנים לחלוטין** – שליש מכל מחלקה. לעובדה זו יש משמעות ישירה בבחירת מדד האיכות: מכיוון שמדובר בבעיה מרובת מחלקות ללא מחלקה מרכזית אחת, מדד האיכות המתאים לפי דרישות המטלה הוא **Macro-Average F1**, שמחשב F1 עבור כל מחלקה בנפרד וממצע ביניהם במשקל שווה – כך שכשל בזיהוי מחלקה אחת אינו "מתחבא" מאחורי הצלחה בשתי האחרות.

1.3 שימוש בכלי AI ובעזרים נוספים

נעזרנו ב-Gemini AI בשלושה שלבים. להלן הפרומפטים ששימשו אותנו ומטרת כל אחד מהם:

מטרה	הפרומפט שבו השתמשנו
בניית שלד ה-Pipeline	I have a Rock-Paper-Scissors image dataset in Google Drive, already split into" train/test folders with one sub-folder per class. Write a clean Google Colab pipeline that mounts the Drive, loads both sets separately without re-splitting them, and prints ".how many images were loaded from each set
הנדסת מאפיינים	The hands are photographed on a plain white background. Suggest an OpenCV" preprocessing function that isolates the hand shape from the background and returns a small fixed-size grid suitable for a distance-based classifier. Explain why resizing a ".non-square bounding box directly distorts the shape, and how to avoid it
כיוון היפר-פרמטרים	Show me how to run GridSearchCV over a KNN classifier with different values of K," distance metrics and weighting schemes, using 5-Fold cross validation and macro F1 as the scoring function, and how to present cv_results_ as a sorted pandas ".DataFrame

בכל אחד מהמקרים השתמשנו בפלט כנקודת פתיחה בלבד: קראנו את הקוד, התאמנו אותו לנתונים שלנו (גודל התמונה, שמות הקטגוריות, טווח הפרמטרים), ובדקנו את התוצאה ויזואלית לפני שהמשכנו הלאה. בפרט, את רכיב ה--Aspect Ratio Padding הוספנו בעצמנו לאחר שראינו במו עינינו שתמונות "נייר" נמעכות בהקטנה ומקבלות צורה הדומה ל"אבן".

```
# -----
# PART 0: Imports
# -----
from google.colab import drive
import os
import cv2
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.metrics import f1_score
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import GridSearchCV, KFold
from sklearn.utils import shuffle
```

```
# -----
# PART 1: Mount Drive & Set Correct Paths
# -----
drive.mount('/content/drive')

base_dir = '/content/drive/MyDrive/ML_Project/Rock_Paper_Scissors/Rock-Paper-Scissors'
train_dir = os.path.join(base_dir, 'train')
test_dir = os.path.join(base_dir, 'test')

categories = ['rock', 'paper', 'scissors']
IMG_SIZE = 48
```

הפרמטר `IMG_SIZE = 48` קובע את גודל הרשת הסופית שאליה מוקטנת כל תמונה לאחר העיבוד. הוא נבחר כפשרה: גדול מספיק כדי לשמר את קווי המתאר של האצבעות, וקטן מספיק כדי שווקטור המאפיינים יישאר בגודל $48 \times 48 = 2,304$ ערכים – סדר גודל שבו KNN עדיין מתפקד היטב.

```
# -----
# PART 2: Load Dataset & Shuffle
# -----
def load_data(directory):
    """Load images and labels from a given directory with early resizing and shuffling."""
    X, y = [], []
    for category in categories:
        folder_path = os.path.join(directory, category)
        if not os.path.exists(folder_path):
            print(f"Warning: Folder not found -> {folder_path}")
            continue

        label = categories.index(category)
        for img_name in os.listdir(folder_path):
            img_path = os.path.join(folder_path, img_name)
            img = cv2.imread(img_path)
            if img is not None:
                # Convert to RGB for proper plotting and resize early to save RAM
                img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
                img_resized = cv2.resize(img_rgb, (150, 150))
                X.append(img_resized)
                y.append(label)

    # Shuffle indices to prevent sequence bias and maintain exact mapping
    indices = np.arange(len(X))
    np.random.shuffle(indices)
    return np.array([X[i] for i in indices]), np.array([y[i] for i in indices])

print("Loading Train and Test datasets separately...")
X_train_raw, y_train = load_data(train_dir)
X_test_raw, y_test = load_data(test_dir)
print(f"Loaded {len(X_train_raw)} Train images and {len(X_test_raw)} Test images.")
```

פלט:

```
Loading Train and Test datasets separately...
Loaded 2520 Train images and 372 Test images.
```

שתי הערות חשובות על הפונקציה:

- **הפרדה מלאה בין הסטים:** הפונקציה מופעלת פעמיים בנפרד, פעם על `train_dir` ופעם על `test_dir`. אין בשום שלב איחוד של הנתונים או חלוקה מחדש, ולכן אין זליגת מידע מסט הבדיקה אל האימון.
- **ערבוב פנימי (Shuffling):** התמונות נקראות מהדיסק לפי סדר התיקיות, כלומר כל ה-`rock` ואז כל ה-`paper` ואז כל ה-`scissors`. ערבוב האינדקסים מונע הטיית רצף – בעיקר בשלב ה-`Cross Validation`, שבו חלוקה לא מעורבת עלולה הייתה ליצור `Fold` שמכיל מחלקה אחת בלבד. שימו לב שהערבוב מופעל על מערך אינדקסים משותף ל-`X` ול-`y`, כך שההתאמה בין תמונה לתווית נשמרת במדויק.

2.1 המוטיבציה

KNN מסווג לפי **מרחק** בין וקטורי מאפיינים. אילו היינו מזינים לו את הפיקסלים הגולמיים, המרחק בין שתי תמונות היה נשלט על ידי גורמים חסרי משמעות לבעיה – גוון העור, בהירות הרקע ומיקום היד בפריים – ולא על ידי צורת התנועה. לכן בנינו פונקציית עיבוד מקדים ייעודית שממירה כל תמונה ל**סילואט בינארי מנורמל** של כף היד. הפונקציה מבצעת חמישה שלבים:

1. **המרה לגונוי אפור** – ביטול תלות בצבע ובגוון העור, וצמצום מ-3 ערוצים לערוץ אחד.
2. **טשטוש גאוסיני (5×5)** – החלקת רעש והסרת שוליים חדים מלאכותיים, כדי שהסך שיחושב בשלב הבא יהיה יציב.
3. **Otsu Thresholding** – חישוב **אוטומטי** של סף ההפרדה בין היד לרקע. Otsu בוחר את הסף שממקסם את השונות בין שתי הקבוצות, ולכן הוא מתאים את עצמו לכל תמונה בנפרד ואינו מצריך קביעת ערך קבוע ידנית. השימוש ב- `THRESH_BINARY_INV` נובע מכך שהרקע בהיר והיד כהה ממנו, ולכן היפוך הסף נותן יד לבנה (255) על רקע שחור (0).
4. **חיתוך קופסה תוחמת (Bounding Box)** – `cv2.findNonZero` מאתר את כל פיקסלי היד, ו- `cv2.boundingRect` מחזיר את המלבן המינימלי שעוטף אותם. החיתוך מסלק את הרקע הריק ומבטל את התלות במיקום היד בתוך הפריים.
5. **ריפוד לריבוע (Aspect-Ratio Padding)** – זהו החלק הקריטי ביותר. קופסה תוחמת של "נייר" היא מלבן ארוך וצר, ואילו של "אבן" היא כמעט ריבועית. הקטנה ישירה של שתיהן ל-48×48 הייתה **מועכת** את הנייר ומשטחת את הפרופורציות שלו, כך שהוא היה נראה למודל דומה מאוד לאבן. לכן אנו מוסיפים שוליים שחורים סימטריים עד שהקופסה הופכת לריבוע מושלם, ורק אז מקטינים. כך יחס הגובה-רוחב המקורי נשמר, והפרופורציה עצמה הופכת למאפיין מבחין.

```
# -----
# PART 3: Feature Engineering (Aspect-Ratio Preserving Crop)
# -----
def get_hand_mask_and_crop(img_rgb):
    """
    Applies Grayscale -> Gaussian Blur -> Otsu Thresholding -> Aspect-Ratio Preserving Crop.
    Padding the cropped bounding box to a perfect square before resizing prevents
    shape distortion (e.g., stopping 'Paper' from squashing and looking like 'Rock').
    """
    gray = cv2.cvtColor(img_rgb, cv2.COLOR_RGB2GRAY)
    blur = cv2.GaussianBlur(gray, (5, 5), 0)

    # Otsu thresholding separates the hand from the white background
    _, thresh = cv2.threshold(blur, 0, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)

    # Find bounding box to crop out empty background
    points = cv2.findNonZero(thresh)
    if points is not None:
        x, y, w, h = cv2.boundingRect(points)
        cropped_hand = thresh[y:y+h, x:x+w]

        # Pad the image to a square BEFORE resizing to preserve aspect ratio
        max_dim = max(w, h)
        pad_top = (max_dim - h) // 2
        pad_bottom = max_dim - h - pad_top
        pad_left = (max_dim - w) // 2
        pad_right = max_dim - w - pad_left

        # Add black padding (value=0) around the hand
        square_hand = cv2.copyMakeBorder(cropped_hand, pad_top, pad_bottom, pad_left, pad_right,
cv2.BORDER_CONSTANT, value=0)
    else:
        square_hand = thresh

    # Resize to the final grid
    resized = cv2.resize(square_hand, (IMG_SIZE, IMG_SIZE))
    return resized

print("\n--- Feature Engineering (Smart Square Crop) ---")
X_train_masks = np.array([get_hand_mask_and_crop(img) for img in X_train_raw])
X_test_masks = np.array([get_hand_mask_and_crop(img) for img in X_test_raw])

# Visualizing the Feature Engineering Process
num_examples = 5
fig, axes = plt.subplots(2, num_examples, figsize=(15, 6))

for i in range(num_examples):
    # Top row: Original images
    axes[0, i].imshow(X_train_raw[i])
    axes[0, i].set_title(f"Original: {categories[y_train[i]].capitalize()}")
    axes[0, i].axis('off')

    # Bottom row: Processed masks
    axes[1, i].imshow(X_train_masks[i], cmap='gray')
    axes[1, i].set_title("Square Cropped Mask")
    axes[1, i].axis('off')

plt.suptitle("Feature Engineering: Original vs. Aspect-Ratio Preserving Crop", fontsize=16,
```

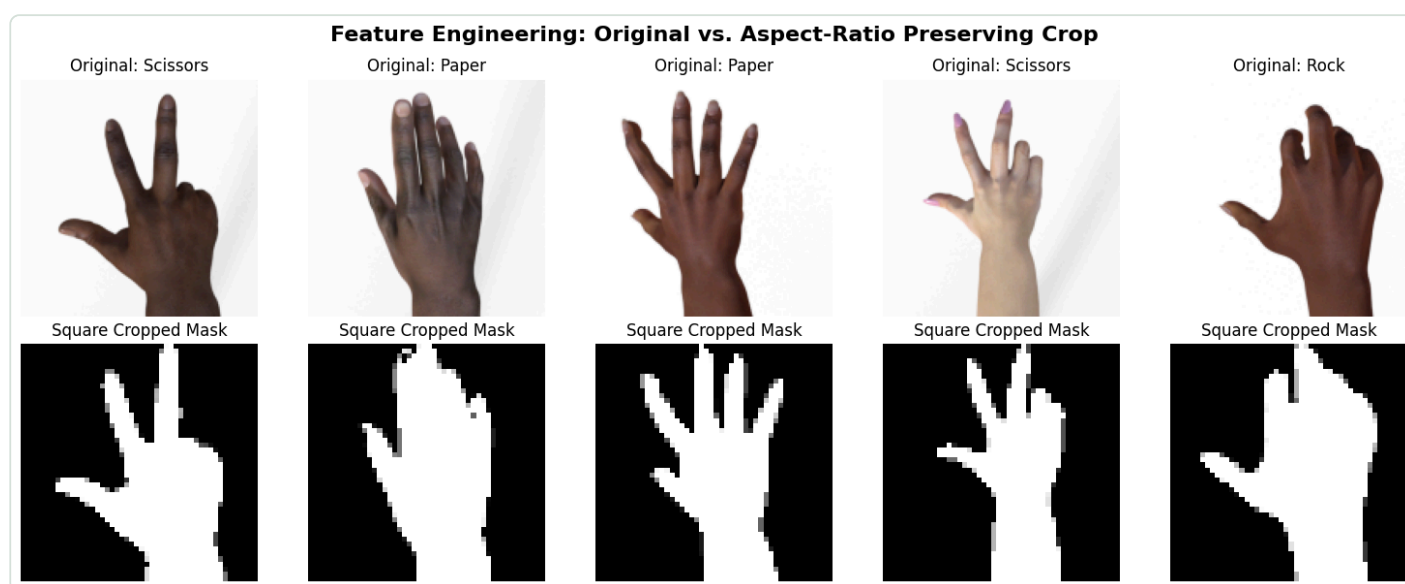
```
fontweight='bold')
plt.tight_layout()
plt.show()

# Flatten and Normalize for the Algorithm
X_train_features = X_train_masks.reshape(len(X_train_masks), -1)
X_test_features = X_test_masks.reshape(len(X_test_masks), -1)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train_features)
X_test_scaled = scaler.transform(X_test_features)
```

2.3 הדגמה ויזואלית של תהליך

בהתאם לדרישת המטלה, אנו מציגים את תהליך הנדסת המאפיינים על דוגמאות מהנתונים. השורה העליונה מציגה את התמונות הגולמיות, והשורה התחתונה את המסכה הבינארית שהתקבלה עבור אותה תמונה בדיוק.



איור א': תהליך הנדסת המאפיינים על חמש דוגמאות מסט האימון – תמונה מקורית (למעלה) מול המסכה הריבועית לאחר העיבוד (למטה)

ניתן לראות בבירור שהעיבוד השיג את מטרתו: הרקע נעלם לחלוטין, גוון העור אינו משפיע עוד על הייצוג, והצורה הגיאומטרית של האצבעות נשמרה. חמש הדוגמאות מכסות את שלוש המחלקות, ודוגמת ה-Paper שבה האצבעות פרושות ממחישה היטב את חשיבות הריפוד – כף היד נותרה רחבה ומלאה גם לאחר ההקטנה ל-48×48, במקום להימנע לצורה מרובעת. מאלף להשוות אותה לדוגמת ה-Paper השנייה, שבה האצבעות צמודות יותר והמסכה שנוצרת צרה בהרבה – בדיוק המקרה שיתברר בהמשך כמקור העיקרי לשגיאות המודל.

הערה על אותו עיבוד בדיוק בסט הבדיקה: הפונקציה `get_hand_mask_and_crop` מופעלת בשורה אחת גם על `X_train_raw` וגם על `X_test_raw`. בכך אנו מבטיחים שסט הבדיקה עובר את אותה טרנספורמציה בדיוק, ללא כל התאמה לנתוני הבדיקה.

2.4 שיטוח ונרמול

לאחר העיבוד, כל תמונה היא מטריצה בגודל 48×48. אנו משטחים אותה לווקטור באורך 2,304 ומעבירים אותה דרך `StandardScaler`. חשוב להדגיש נקודה מתודולוגית: ה-`Scaler` מותאם (`fit_transform`) אך ורק על נתוני

האימון, ועל נתוני הבדיקה מופעל `transform` בלבד. חישוב הממוצע וסטיית התקן על סמך סט הבדיקה היה מהווה זליגת נתונים.

חלק 3 – אלגוריתם הלמידה: K-Nearest Neighbors

3.1 כיצד האלגוריתם עובד

KNN הוא אלגוריתם **עצל (Lazy Learner)**: בשלב האימון הוא אינו בונה מודל פרמטרי ואינו מבצע אופטימיזציה, אלא רק שומר בזיכרון את כל דוגמאות האימון. כל העבודה מתרחשת בזמן החיזוי – עבור תמונה חדשה, האלגוריתם מחשב את המרחק שלה מכל 2,520 תמונות האימון, בוחר את K השכנות הקרובות ביותר, ומשייך אותה למחלקה שקיבלה את מירב ה"קולות" מבין השכנות הללו.

ההיפר-פרמטרים שאותם אפשרנו לכוון, בהתאם לנלמד בכיתה:

היפר-פרמטר	משמעות והשפעה
<code>n_neighbors (K)</code>	מספר השכנים המשתתפים בהצבעה. K קטן יוצר גבול החלטה גמיש ורגיש לרעש (נטייה ל-Overfitting), ואילו K גדול ממצע על פני אזור רחב ומחליק את גבול ההחלטה (נטייה ל-Underfitting).
<code>metric</code>	אופן מדידת המרחק. Euclidean (L2) הוא המרחק הישר במרחב, ורגיש יותר להפרשים גדולים בודדים. Manhattan (L1) מסכם את ההפרשים המוחלטים לאורך כל הצירים, ולרוב עמיד יותר במרחבים בעלי ממד גבוה כמו שלנו.
<code>weights</code>	Uniform – לכל השכנים משקל זהה בהצבעה. Distance – משקל כל שכן הוא ביחס הפוך למרחקו, כך שדוגמה דומה מאוד משפיעה יותר מדוגמה גבולית.

3.2 פונקציות האימון והחיזוי

המימוש נשען על `KNeighborsClassifier` מ-`scikit-learn`, החושף בדיוק את שתי הפונקציות הנדרשות במטלה:

- אימון** – `fit(X_train_scaled, y_train)`. במקרה של KNN, פעולה זו בונה מבנה נתונים לחיפוש יעיל של שכנים (KD-Tree / Ball-Tree) מעל דוגמאות האימון.
- חיזוי** – `predict(X_test_scaled)`. הפונקציה מחזירה עבור כל דוגמה את התווית שזכתה בהצבעת K השכנים, לפי המטריקה והמשקולות שנבחרו.

בקוד, קריאת ה-`fit` מתבצעת דרך `grid_search.fit(...)` (המפעילה אותה עבור כל צירוף פרמטרים, ולבסוף מאמנת מחדש את המודל המנצח על כל סט האימון), וקריאת ה-`predict` מתבצעת בחלק 5 על המודל הנבחר.

חלק 4 – אימון: Grid-Search ו-Cross Validation

4.1 השיטה

כדי לא לבחור היפר-פרמטרים באופן שרירותי, וכחלק מסעיף ההרחבות, ביצענו סריקה שיטתית של **המכפלה הקרטזית** של כל טווחי הפרמטרים: 5 ערכים של K , על 2 שיטות שקלול, על 2 מטריקות מרחק – סה"כ **20 צירופים**. כל צירוף הוערך

תחת **Fold Cross Validation-5**: סט האימון מחולק לחמישה חלקים שווים, המודל מאומן חמש פעמים על ארבעה חלקים ונבדק על החלק שנותר, והציון הסופי הוא הממוצע של חמש ההרצות. סה"כ בוצעו 100 אימונים. השימוש ב- `random_state=42` מבטיח שהחלוקה לקפלים ניתנת לשחזור, ומדד הניקוד שהוגדר הוא `f1_macro` – בדיוק המדד שלפיו נמדדת המטלה.

4.2 הקוד

```
# -----
# PART 4: Algorithm Implementation, Grid Search & Training
# -----
print("\n--- Hyperparameter Tuning (Grid Search & 5-Fold CV) ---")

# Define the hyperparameter grid to search (Cartesian product)
param_grid = {
    'n_neighbors': [5, 11, 15, 19, 25],
    'weights': ['uniform', 'distance'],
    'metric': ['euclidean', 'manhattan']
}

knn = KNeighborsClassifier()

# Configure 5-Fold Cross Validation
cv_strategy = KFold(n_splits=5, shuffle=True, random_state=42)

# Initialize GridSearchCV using Macro F1-Score as the scoring metric
grid_search = GridSearchCV(
    estimator=knn,
    param_grid=param_grid,
    cv=cv_strategy,
    scoring='f1_macro',
    n_jobs=-1
)

# Execute Grid Search
print("Running Grid Search... This might take a minute.")
grid_search.fit(X_train_scaled, y_train)

# Presenting Results in a DataFrame (Bonus Requirement)
results_df = pd.DataFrame(grid_search.cv_results_)
display_cols = ['param_n_neighbors', 'param_weights', 'param_metric', 'mean_test_score']
results_df = results_df[display_cols].sort_values(by='mean_test_score', ascending=False)
results_df.columns = ['K (Neighbors)', 'Weights', 'Distance Metric', 'Mean CV Macro F1']

print("\n>>> Top 5 Hyperparameter Permutations (Cross-Validation Results DataFrame):")
print("=====")
print(results_df.head(5).to_string(index=False))
print("=====")

print(f"\n>>> BEST PERMUTATION FOUND:")
print(f"Parameters: {grid_search.best_params_}")
print(f"Best Macro Average F1-Score (Cross-Validation): {grid_search.best_score_:.4f}")

# Extract the best model (GridSearchCV automatically refits it on the full trainset)
best_knn_model = grid_search.best_estimator_
print("\nTraining final model on the entire trainset using the best parameters... Completed.")
```

```
>>> Top 5 Hyperparameter Permutations (Cross-Validation Results DataFrame):
=====
K (Neighbors)  Weights Distance Metric  Mean CV Macro F1
              5 distance      manhattan      0.999598
              5 distance      euclidean      0.999222
              5 uniform      manhattan      0.999206
              5 uniform      euclidean      0.998830
             11 distance      manhattan      0.998411
=====

>>> BEST PERMUTATION FOUND:
Parameters: {'metric': 'manhattan', 'n_neighbors': 5, 'weights': 'distance'}
Best Macro Average F1-Score (Cross-Validation): 0.9996

Training final model on the entire trainset using the best parameters... Completed.
```

חמשת הצירופים המובילים, כפי שהוצגו בטבלת ה-DataFrame:

Mean CV Macro F1	Distance Metric	Weights	K
0.999598	manhattan	distance	5
0.999222	euclidean	distance	5
0.999206	manhattan	uniform	5
0.998830	euclidean	uniform	5
0.998411	manhattan	distance	11

הצירוף המנצח: {'metric': 'manhattan', 'n_neighbors': 5, 'weights': 'distance'} עם ציון Macro F1 של **0.9996** ב-Cross Validation. שתי מגמות ברורות עולות מהטבלה. הראשונה: **ככל ש-K קטן יותר הציון גבוה יותר** – כל ארבעת הצירופים המובילים הם K=5, ורק החמישי הוא K=11. זו עובדה שנחזור אליה בניתוח התוצאות. השנייה: **כל חמשת הצירופים המובילים צמודים זה לזה** – כולם בטווח שבין 0.9984 ל-0.9996, כלומר הפרש של פחות מ-0.002. המשמעות היא שבחירת ההיפר-פרמטרים כמעט אינה משנה כאן, ומה שקבע בפועל את איכות המודל הוא העיבוד המקדים שתואר בחלק 2.

לאחר סיום הסריקה, GridSearchCV מבצע refit אוטומטי: הוא מאמן מחדש את המודל עם הפרמטרים המנצחים על כל 2,520 דוגמאות האימון (ולא רק על 4/5 מהן כפי שקרה בתוך הקפלים). מודל זה, best_knn_model, הוא המודל היחיד שמופעל בהמשך על סט הבדיקה.

```

# -----
# PART 5: Prediction and Evaluation on Test Set
# -----
print("\n--- Prediction and Model Quality Evaluation ---")

if len(X_test_scaled) > 0:
    # Predict using the BEST model found by Grid-Search
    y_pred = best_knn_model.predict(X_test_scaled)

    # Classification Visualization: 5 Rows x 5 Columns (25 Images)
    rows, cols = 5, 5
    fig, axes = plt.subplots(rows, cols, figsize=(18, 18))

    for row_idx in range(rows):
        for col_idx in range(cols):
            img_idx = row_idx * cols + col_idx

            if img_idx < len(X_test_raw):
                ax = axes[row_idx, col_idx]
                ax.imshow(X_test_raw[img_idx])

                actual_label = categories[y_test[img_idx]].capitalize()
                pred_label = categories[y_pred[img_idx]].capitalize()

                text_color = 'green' if y_test[img_idx] == y_pred[img_idx] else 'red'
                title_text = f"Actual: {actual_label}\nPred: {pred_label}"

                ax.set_title(title_text, color=text_color, fontweight='bold', fontsize=12)
                ax.axis('off')

            # Add a dashed line separator between rows
            if row_idx < rows - 1:
                ax_mid = axes[row_idx, 2]
                ax_mid.plot([-2.5, 3.5], [-0.3, -0.3], color='gray', linestyle='--', linewidth=2,
transform=ax_mid.transAxes, clip_on=False)

    plt.suptitle("25 Diverse Predictions vs Actual (Test Set)", fontsize=22, fontweight='bold')
    plt.tight_layout(rect=[0, 0, 1, 0.96])
    fig.subplots_adjust(hspace=0.8)
    plt.show()

    # Calculate Final Macro Average F1-Score
    macro_f1 = f1_score(y_test, y_pred, average='macro')
    print(f"\n=====")
    print(f"Final Model Quality on Test Set (Macro Average F1-Score): {macro_f1:.4f}")
    print(f"=====")
else:
    print("Test set is empty. Cannot perform evaluation.")

```

25 Diverse Predictions vs Actual (Test Set)



איור ב': חיזוי מול תוויית אמת על עשרים וחמש דוגמאות מסט הבדיקה – ירוק = סיווג נכון, אדום = סיווג שגוי

מדד האיכות הסופי:

Final Model Quality on Test Set (Macro Average F1-Score): 0.7738

ציון ה-Macro-Average F1 של המודל על 372 תמונות הבדיקה הוא **0.7738**.

מדוע הציון ב-Cross Validation היה 0.9996 והציון על ה-Test רק 0.7738? הפער אינו באג אלא תופעה שיש לה הסבר ברור, והוא הממצא המעניין ביותר בעבודה. ה-Grid-Search בוצע כולו **בתוך סט האימון בלבד**. תמונות ה-Dataset צולמו בסדרות (Bursts) – עשרות פריימים רצופים של אותה יד באותה תנוחה, עם הבדלים מזעריים ביניהם. כאשר סט האימון מחולק לחמישה קפלים, פריימים "תאומים" מאותה סדרה מתפזרים בין קפל האימון לקפל הוולידציה. מכיוון ש-KNN מחפש את השכן הקרוב ביותר, הוא מוצא בקפל האימון תמונה כמעט זהה לתמונה שהוא אמור לחזות – ולכן משיג ציון מושלם. זה גם מסביר מדוע דווקא $K=5$ ו-`weights='distance'` ניצחו: ככל ש-K קטן יותר, כך גדל המשקל של אותם "תאומים" קרובים במיוחד.

לעומת זאת, סט הבדיקה מכיל **סדרות צילום שונות לחלוטין**, שאף פריימים מהן לא הופיעו באימון. כאן נעלם קיצור הדרך, והמודל נאלץ להסתמך על דמיון צורני אמיתי בלבד. לכן **0.7738 הוא הביצוע הריאלי של המודל**, וה-0.9996 הוא אינדיקציה ל-Overfitting אופייני של KNN על נתונים חופפים.

ניתוח השגיאות: מבין עשרים וחמש הדוגמאות המוצגות באיור ב', שמונה עשויה סווגו נכון ושבע שגו – **וכל שבע השגיאות, ללא יוצא מן הכלל, הן תמונות Paper:** ארבע מהן סווגו כ-Rock ושלוש כ-Scissors. התמונה המלאה חדה עוד יותר כאשר סופרים מהכיוון ההפוך – מתוך שמונה תמונות Paper שמופיעות באיור, רק אחת סווגה נכון. לעומת זאת, כל תמונות ה-Rock וה-Scissors שבאיור סווגו נכון.

הכיוון הזה עקבי ואינו מקרי, והוא נובע ישירות מהייצוג שבחרנו. "אבן" היא גוש מלא וקומפקטי, ו"מספריים" היא צורה עם שתי אצבעות בולטות – שתיהן חתימות ברורות ויציבות. "נייר", לעומת זאת, היא המחלקה הרגישה ביותר לזווית הצילום: כאשר כף היד מצולמת בזווית או כשהאצבעות צמודות זו לזו, המסכה הבינארית מאבדת את הרווחים ביניהן. אם האצבעות נשארות פרושות מעט, מה שנותר דומה ל"מספריים"; ואם הן נצמדות לגמרי, הסילואט מתכווץ לגוש מלא ודומה ל"אבן" – וזה בדיוק מה שרואים באיור. **Paper היא המחלקה החלשה של המודל**, ובדיוק בשל כך Macro-F1 הוא המדד הנכון כאן: הוא אינו מאפשר לכשל במחלקה אחת להיטשטש מאחורי הצלחה כמעט מלאה בשתי האחרות. לו היינו מודדים ב-Accuracy בלבד, החולשה הזו הייתה כמעט נעלמת מהעין.

סיכום ומסקנות

בנינו Pipeline מלא של למידה מונחית לבעיית סיווג תמונות מרובת מחלקות: טעינה מופרדת של Train ו-Test, הנדסת מאפיינים ייעודית מבוססת Otsu וריפוד שומר-פרופורציות, כיוון היפר-פרמטרים ב-Grid-Search תחת Fold Cross-5 Validation, ואימון מחדש של הצירוף המנצח על כל סט האימון. הציון הסופי על נתונים שהמודל לא ראה מעולם הוא **Macro-F1 = 0.7738**.

מה למדנו:

- **הנדסת המאפיינים היא שקבעה את התוצאה, לא בחירת האלגוריתם.** KNN אינו "רואה" צורות אלא רק מרחקים; המעבר מפיקסלים גולמיים לסילואט ריבועי מנרמל הוא מה שהפך את הבעיה לפתירה עבורו.
- **ציון ולידציה גבוה אינו בהכרח מודל טוב.** ה-0.9996 שקיבלנו ב-Cross Validation נבע ממבנה הנתונים (פריימים חופפים) ולא מיכולת הכללה. רק ההפרדה המלאה של סט הבדיקה חשפה את הביצוע האמיתי.
- **לשגיאות יש מבנה.** השגיאות אינן מפוזרות אקראית אלא מרוכזות בכיוון אחד (Paper→Scissors), ומצביעות על מגבלה ספציפית של הייצוג שבחרנו.

כיווני שיפור אפשריים:

- חלוקת ה-Cross Validation לפי **קבוצות** (GroupKFold לפי סדרת צילום) כדי שהציון בוולידציה ישקף את הביצוע האמיתי כבר בשלב הכיוון.
- הוספת מאפייני צורה מפורשים – מספר האצבעות הבולטות, יחס שטח היד לשטח הקופסה התוחמת (Extent), או קמירות (Convexity Defects) – שיפרידו בבירור בין Paper ל-Scissors.
- הרחבת נתונים (Augmentation) בסיבובים קלים, כדי להקטין את רגישות המודל לזווית הצילום.
- מעבר לאלגוריתם המסוגל ללמוד ייצוג מרחבי בעצמו, כגון רשת נוירונים קונבולוציונית (CNN).